

Development Plan Flexin

Team 3, Flexin
Arjun Karthik
Shao (Daniel) Hu
Noah Goldschmied
Peter Deforest
Daniel Franze-Da Silva

Table 1: Revision History

Date	Developer(s)	Change
Date1	Name(s)	Description of changes
Date2	Name(s)	Description of changes
...	...	...

[Put your introductory blurb here. Often the blurb is a brief roadmap of what is contained in the report. —SS]

[Additional information on the development plan can be found in the [lecture slides](#). —SS]

1 Confidential Information?

[State whether your project has confidential information from industry, or not. If there is confidential information, point to the agreement you have in place. —SS]

[For most teams this section will just state that there is no confidential information to protect. —SS]

2 IP to Protect

[State whether there is IP to protect. If there is, point to the agreement. All students who are working on a project that requires an IP agreement are also required to sign the “Intellectual Property Guide Acknowledgement.” —SS]

3 Copyright License

[What copyright license is your team adopting. Point to the license in your repo. —SS]

4 Team Meeting Plan

[This section is for meetings between the team members, and meetings between the team and their supervisor/advisor (as appropriate). —SS]

[How often will the team meet? where? when? —SS]

[If the meeting is a physical location (not virtual), out of an abundance of caution for safety reasons you shouldn’t put the location online —SS]

[To help schedule meetings, you can use the lecture/tutorial time when we do not have anything else scheduled. —SS]

[How often will you meet with your supervisor/advisor? when? where? —SS]

[Will meetings be virtual? At least some meetings should likely be in-person. —SS]

[How will the meetings be structured? There should be a chair for all meetings. There should be an agenda for all meetings. —SS]

5 Team Communication Plan

The following outlines the communication tools that will be used throughout the project:

- **GitHub:** a main repository will be used for version control and issue tracking. As well, a separate GitHub project will be used for project management, where issues will be tracked using a Kanban board with 4 states: To Do, In Progress, In Review, and Done.
- **Microsoft Teams:** Microsoft Teams will be used as the main source of communication between team members. A Microsoft Teams group chat will be used for adhoc messaging and communication. A Microsoft Teams Channel will be used for conducting virtual team meetings, storing any shared files that do not belong in the code repository, and posting communication regarding pull requests and their reviews.
- **Email:** our McMaster emails will be used for communication with our supervisor/TA and professors.

6 Team Member Roles

All team members will be responsible for contributing to the development of the project (this includes coding, documentation, testing, reviewing, etc.). In addition to this contribution, each team member will be assigned the following roles (roles can be modified if necessary):

Table 2: Team Member Role Assignment

Team Member	Role(s)	Responsibilities
Daniel Franze-Da Silva	Scribe	Take thorough notes during meetings
	UI/UX Design Lead	In charge of ensuring the UI and UX meet the high standards of the end users
Noah Goldschmied	Team Liason	Act as the main point of contact for the team when communicating with external individuals relevant to the project (i.e. TAs, supervisors, professors, potential partnering companies, etc.)
	Chairman	Lead team meetings, set the agenda for upcoming meetings and guide discussions
Arjun Karthik	Github Manager	In charge of managing the GitHub repository, managing and creating issues, and ensuring that the GitHub project is organized and complete
	CI/CD Lead	In charge of ensuring that the CI/CD pipeline is set up and functioning properly, and that all team members are adhering to the regulations put into place (i.e. branching, pull requests and approvals, proper naming conventions, etc.)
Peter Deforest	Testing Lead	In charge of ensuring that proper tests are conducted, regression testing is upheld when new logic is introduced, and that the testing process is properly integrated into the development
Shao (Daniel) Hu	Lead Developer	In charge of leading the development efforts and technical design, and helping teammates overcome technical roadblocks

7 Workflow Plan

The project workflow will be managed using Git, with our code and documentation hosted in our GitHub repository. Our **main** branch will serve as the primary codebase and will be the branch used for product deployments. To make contributions, developers are required to create branches directly from **main** and commit any initial changes to their respective branch.

7.1 Branching Strategy

We will utilize the following branch categories:

- **feat (feature)**: For development on a new feature.
- **bugfix**: For addressing and fixing identified issues within the codebase.
- **chore**: For code changes that support later development, such as refactoring or clean-up work.

Additionally, each branch must have a descriptive name that clearly indicates which issue in the GitHub project is being addressed.

7.2 Issue Tracking and Project Management

Issues will be categorized based on urgency, from **P0** (highest priority) to **P2** (lowest priority), ensuring that critical tasks are addressed promptly.

The GitHub project board has been configured with the following setup: **Backlog** → **In Progress** → **Review** → **Closed**. As development progresses, it is the developer's responsibility to keep their assigned issues updated to maintain team visibility on project status.

7.3 Pull Requests and CI/CD Pipeline

As part of the new code added to a branch, developers are required to implement test cases to ensure adequate code coverage. Once code is committed, a Pull Request (PR) must be raised to merge the branch into **main**.

Our Continuous Integration/Continuous Deployment (CI/CD) pipeline will enforce the following checks before a merge is permitted:

- Automated testing and deployment checks.
- Linter execution to ensure code meets formatting and quality standards.
- Verification that the feature branch is up-to-date with **main**.

7.4 Code Review Process

In addition to passing all CI/CD checks, each PR requires a manual review and approval from at least one other group member. The PR comments section will be utilized to trace the iterative process of feedback and the subsequent code changes made to address said feedback.

7.5 Pull Request Template Requirements

Each PR must be directly linked to its corresponding issue in the GitHub Project board to maintain traceability. Furthermore, all PRs must adhere to a standardized template that includes:

- A comprehensive summary of the changes made.
- Sample test cases and step-by-step instructions for the reviewer to verify the code.
- Screenshots of the updated UI (if the PR includes user interface modifications).

8 Project Decomposition and Scheduling

For aiding in documenting and tracing our project deliverables, we will be making use of two GitHub projects that will handle our code and our documentation deliverables. They are linked below:

- **Code Project:** <https://github.com/users/arjunkarthik123/projects/4/views/1>
- **Documentation Project:** <https://github.com/users/arjunkarthik123/projects/3>

Both projects are split into four sections: Backlog, In Progress, In Review, and Done. As the project progresses, we will come up with items that need to be addressed for both our documentation and later on our codebase. Whenever any team member thinks of an item that needs to be worked on, it will be added to the backlog of the respective projects. Depending on what the team decides, someone will be assigned the new issue, and whenever the person works on it, it will be under the In Progress section. In order to complete the deliverable, it first needs to be reviewed, hence the In Review section. For either the code or documentation, a pull request will be created and linked to the respective issue being addressed, and only when it gets approved will the issue move to the Done section.

If the issues seem to be larger than anticipated, we will split up the issue using sub-issues and have them assigned accordingly. This is to ensure no single issue is too big to handle by one person alone.

Table 3 provides a high-level overview of the key deliverables throughout the academic year (please note that these items were taken directly from the course schedule in the course outline):

Table 3: Key Deliverables Schedule

Term	Week	Date	Tasks / Deliverables
Term 1	Week 2	Week of Sep 14, 2026	Deliverable: Deadline for teams formation, projects selection (end of week)
Term 1	Week 3	Week of Sep 21, 2026	Deliverable: Biosheets due to Avenue dropbox TA Meeting: Review of prob state, PoC plan, dev plan and team charter
Term 1	Week 4	Week of Sep 28, 2026	Deliverable: Problem Statement, Proof of Concept Plan, Development Plan, Team Charter Rev 0 Peer Review: Prob State + PoC plan + Dev Plan + Team Charter peer review
Term 1	Week 5	Week of Oct 05, 2026	TA Meeting: Review SRS+Haz Analysis
Term 1	Week 6	Week of Oct 19, 2026	Deliverable: SRS + Haz Analysis Rev 0 Peer Review: SRS + Haz Analysis peer review
Term 1	Week 7	Week of Oct 26, 2026	TA Meeting: Review VnV Plan
Term 1	Week 8	Week of Nov 02, 2026	Deliverable: VnV Plan Rev 0 Peer Review: VnV Plan peer review
Term 1	Week 9	Week of Nov 09, 2026	TA Meeting: Review Des Doc Rev-1 and CD-Fall
Term 1	Week 10	Week of Nov 16, 2026	Deliverable: Des Doc Rev -1 and CD-Fall Peer Review: Des Doc Rev-1 Peer Review
<i>Continued on next page</i>			

Table 3: Key Deliverables Schedule (Continued)

Term	Week	Date	Tasks / Deliverables
Term 1	Week 11	Week of Nov 23, 2026	Presentation/Demo: Proof of Concept Demos
Term 1	Week 12	Week of Nov 30, 2026	Presentation/Demo: Proof of Concept Demos
Term 1	Week 13	Week of Dec 07, 2026	Presentation/Demo: Code review interviews
Term 2	Week 15	Week of Jan 11, 2027	TA Meeting: Review of Des Doc
Term 2	Week 16	Week of Jan 18, 2027	Deliverable: Design Document Rev 0 Peer Review: Module Implementation due
Term 2	Week 17	Week of Jan 25, 2027	Peer Review: Des Document peer review (module implementation reflection)
Term 2	Week 18	Week of Feb 01, 2027	Presentation/Demo: Rev 0 Demo
Term 2	Week 19	Week of Feb 08, 2027	Presentation/Demo: Rev 0 Demo
Term 2	Week 20	Week of Feb 22, 2027	Presentation/Demo: Code review interviews
Term 2	Week 21	Week of Mar 01, 2027	TA Meeting: Review VnV Report+CD-Winter
Term 2	Week 22	Week of Mar 08, 2027	Deliverable: VnV Report + CD-Winter Peer Review: VnV Report + CD-Winter peer review
Term 2	Week 23	Week of Mar 15, 2027	Presentation/Demo: Final Demos
Term 2	Week 25	Week of Mar 29, 2027	Deliverable: EXPO poster pdf and video due; Final Documentation (Rev 1)
Term 2	Week 26	Week of Apr 05, 2027	Presentation/Demo: Capstone EXPO

9 Proof of Concept Demonstration Plan

What is the main risk, or risks, for the success of your project? What will you demonstrate during your proof of concept demonstration to convince yourself that you will be able to overcome this risk?

10 Expected Technology

[What programming language or languages do you expect to use? What external libraries? What frameworks? What technologies. Are there major components of the implementation that you expect you will implement, despite the existence of libraries that provide the required functionality. For projects with machine learning, will you use pre-trained models, or be training your own model? —SS]

[The implementation decisions can, and likely will, change over the course of the project. The initial documentation should be written in an abstract way; it should be agnostic of the implementation choices, unless the implementation choices are project constraints. However, recording our initial thoughts on implementation helps understand the challenge level and feasibility of a project. It may also help with early identification of areas where project members will need to augment their training. —SS]

Topics to discuss include the following:

- Specific programming language
- Specific libraries
- Pre-trained models
- Specific linter tool (if appropriate)
- Specific unit testing framework
- Investigation of code coverage measuring tools
- Specific plans for Continuous Integration (CI), or an explanation that CI is not being done
- Specific performance measuring tools (like Valgrind), if appropriate
- Tools you will likely be using?

[git, GitHub and GitHub projects should be part of your technology. —SS]

11 Use of AI and LLMs

[This section is a continuation of the previous section. The role of LLMs in your project as important enough to warrant their own section. How will your team be using LLMs for code and documentation development? What tools will you be using? How will you verify the output of your LLMs? —SS]

12 Coding Standard

[What coding standard will you adopt? —SS]

13 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

[This section is about the use of AI to write this document, not your planned use of AI for your project. You discuss that topic in one of the sections in the body of this report. —SS]

13.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

13.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

13.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

Appendix — Reflection

[Not required for CAS 741 —SS]

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. Why is it important to create a development plan prior to starting the project?
2. In your opinion, what are the advantages and disadvantages of using CI/CD?
3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Appendix — Team Charter

External Goals

- Develop a finalized and working solution that we can be proud of and use as a strong project reference on resumes and in interviews.
- Develop a solution in a topic of interest that we enjoy and can relate to, so that we can maintain motivation and genuine enjoyment during development of the solution.
- Achieve a grade of at least an A (11 on McMaster's 12-point scale) in the capstone course
- Make it to the Capstone EXPO and standout positively among the other software projects
- Grow our professional skills, project management skills, and problem solving skills while working through a real world problem

Attendance

Expectations

- All team members are expected to attend all scheduled meetings, unless they have an acceptable excuse for missing the meeting
- Arrive at least 10 minutes before to the agreed upon location if the meeting is held in person, and at arrive at least 5 minutes before any scheduled virtual meetings to ensure preparedness and that the full meeting time may be utilized effectively
- If a team member is unable to attend a meeting, they are expected to notify the rest of the team in the Microsoft Teams groupchat at least 24 hours in advance, unless it is an emergency situation and out of their control
- If a team member misses a meeting or misses part of a meeting, they are expected to review the meeting notes and any relevant documents before the next meeting to get caught up on what they missed
- If a team member misses 2 meetings in a row without an acceptable excuse or proper communication with the rest of the team, the team will schedule an emergency meeting to address this issue. If the issue persists, it will be escalated to the TA and/or instructor for further action
- Arriving late to a meeting or leaving early without an acceptable excuse will result in a warning. If this happens more than 3 times, the team will schedule an emergency meeting to address this issue. If the issue persists, it will be escalated to the TA and/or instructor for further action

Acceptable Excuse

Acceptable Excuses:

- Unforeseen circumstances that are out of the teammate's control (i.e. car breaks down on the way to a meeting)
- Medical appointments that were scheduled in advance and cannot be rescheduled
- Medical, personal, or family emergencies
- Personal illness or injury that hinders the teammate's ability from attending and or providing meaningful contributions

Unacceptable Excuses:

- Technical difficulties during virtual meetings (i.e. internet issues, computer issues, etc.). These can and should be resolved before the meeting starts
- Forgetting about a meeting or forgetting to notify the team about a prior conflict
- Other course deliverables. These meetings are scheduled in advance and thus should be planned around in order to meet deadlines for all courses

In Case of Emergency

1. Contact the rest of the team immediately about the emergency (the specific details of the emergency are not required to be shared as they may be personal)
2. Assess the situation and determine how long this emergency will hinder your progress (i.e. will just miss one meeting, will miss a deliverable deadline, will miss extended time)
3. Find a replacement/backup team member with extra capacity to cover for you
 - (a) Meetings - if you have a prominent role in the meeting(s) (i.e. scribe) find and contact a team member that can fill the role of scribe for the missed meeting(s)
 - (b) Deliverables - if your assigned task is yet to be completed, communicate your progress with the team so they can delegate that responsibility to another team member with extra capacity to complete the task
4. When the emergency is resolved and you are able to get back into the project, catch up on what you have missed

Accountability and Teamwork

Quality

In order to maintain consistent quality in our work, the following are expected of all team members:

- Meeting Preparation
 - Team members are expected to come to meetings fully prepared reading any necessary materials
 - Team members are expected to come to meetings with their assigned tasks completed and ready to discuss their progress with the team
- Deliverables
 - Each deliverable should be thoroughly reviewed by all team members before submission to ensure that it meets the quality standards of the team
 - Team members should strive to produce work that meets the level 4 criteria in each deliverable rubric

Attitude

- Team members are expected and encouraged to provide constructive feedback
- Team members are expected to be positive and respectful when interacting with other team members, TAs, instructors, and any other stakeholders
- Team members should be open to new ideas and willing to compromise when necessary
- Team members should be patient and maintain professionalism when conflict arises
- Team members should feel as though they can openly approach any other team member with questions, concerns, or ideas
- Team members should welcome all ideas for consideration, instead of shutting down ideas without discussion
- Team members should be willing to contribute their fair share of the workload

Stay on Track

- The team will leverage the Kanban board in our GitHub Documentation Project Tracker to track the status of our tasks and deliverables. Each team member is responsible for keeping their assigned issues up-to-date on the Kanban board
- The team will also leverage Avenue to Learn and the Google calendar provided by the instructor to track deadlines for deliverables
- During scheduled lecture time slots (meeting held regardless whether the lecture is running or not - meeting held after lecture if necessary), 'stand-up' style meetings will occur to monitor incremental progress to ensure that all team members stay on track to meet deliverable deadlines
- The team will leverage performance metrics on the Github repository to monitor attendance records, and contributions (i.e. commits, pull requests, etc.)
- No target contribution metrics per deliverable as workload may vary. However, by the conclusion of the project, contributions should be relatively equal among all team members ranging from 20-30% of the total contributions per team member
 - If contribution targets are not met for an individual team member by the end of the project, the team will escalate this issue to the TA and/or instructor to have the capstone grade adjusted accordingly
 - If the team notices the decline in contributions of a team member during the project, the team will adjust future workloads accordingly to balance out contributions throughout the entirety of the project and enforce fairness
- For attendance, it is expected that all team members attend at least 90% of all scheduled meetings

Team Building

- The team will engage in icebreakers and non-project related discussions for a few minutes before each meeting to better get to know each other and build meaningful relationships
- The team will get together after successful deadline submissions and project milestones to celebrate a job well done
- The team will hold social meetings that are not project related at the start of the project, halfway through the project (end of fall term), and at the very conclusion of the project (end of winter term after the Capstone EXPO)

Decision Making

1. All ideas will be considered and discussed by the team before a decision is reached
2. The team will first attempt to come to a consensus on a decision as we should all be in agreement and aligned before making impactful decisions
3. If a consensus cannot be reached, the team will hold a vote on the decision. For an idea to be passed a majority vote of 3 out of 5 members is required
4. If the team is struggling to decide between options (i.e. they cannot clearly decide where to cast their votes), an unbiased third party will be involved to help the team come to a final decision